



Partitioning in PostgreSQL

Partitioning refers to splitting a large table into smaller pieces. This article covers the basics of partitioning in PostgreSQL.

Currently, PostgreSQL supports range and list partitioning via table inheritance. Basically, you have to create each partition as a child table of the master table. Generally, if you want to split data into specific ranges, then use range partitioning. Date and timestamp values are good examples for this. For example, you may want to split yearly data into quarterly partitions. Similarly, if you want to split data based on a specific list of values, then use list partitioning. For this, you can consider creating partitions for each state of a country. Generally, in range partitioning, you would be adding more partitions over time, whereas in list partitioning the data will be continuously growing within each partition. This design decision is very important from the partition maintenance point of view, which most people ignore.

Although the syntax for creating partitioned tables (just like the one in a commercial RDBMS) is not available in PostgreSQL, you may wonder exactly how to set up partitioning in PostgreSQL. You will be glad to know that PostgreSQL uses the concept of inheritance, triggers and constraint exclusion to support partitioning. Additionally, you can also use rules instead of triggers. Now, inheritance and constraint exclusion come with their own pros and cons. There are various scenarios that you need to take care of. Sharing details on them is the main goal of this article. Further, once that has been addressed, the next goal should be to automate partition maintenance because you would most certainly like to automate the creation and maintenance part of partitioning. As you will realise by the end of this

article, partition maintenance is another important task, apart from setting up the partitions.

Partitioning using inheritance

Before we begin, you should note that there are various ways to set up partitioning. The scheme that I am going to show here is just one of many. You would need to tweak it as per the actual requirements. Before we start, make sure that the *constraint_exclusion* configuration option is set to either 'ON' or 'PARTITION' in the *postgresql.conf* file. This is absolutely necessary to get the partitioning scheme working.

First, let us create an empty master table without any primary key. If you are wondering about the missing primary key, then you should know that this is an empty master table, and that we need to set up a primary key on each of the partitions separately (more on this in the latter part of the article):

```
CREATE TABLE orders (
    id          INT NOT NULL,
    address     TEXT NOT NULL,
    order_date  TIMESTAMP NOT NULL
);
```

Now, create some partitions using non-overlapping CHECK constraints:

```
CREATE TABLE orders_part_2011 (
    CHECK (order_date >= DATE '2011-01-01'
           AND order_date < DATE '2012-01-01')
```